

grationsprojectdiscord supportcategoryintegrationsprojectemailsonpush
supportcategoryintegrationsprojectgithub supportcategoryintegrationsprojectgooglechat
supportcategoryintegrationsprojecttinker supportcategoryintegrationsprojectjenkins
supportcategoryintegrationsprojectmattermost supportcategoryintegrationsprojectmsteams
supportcategoryintegrationsprojectstatusemails supportcategoryintegrationsprojectpivotal
supportcategoryintegrationsprojectprometheus supportcategoryintegrationsprojectslack
supportcategoryintegrationsprojectunifycircuit supportcategoryintegrationsprojectwebex
supportcategoryintegrationsprojectwebhooks supportcategoryintegrationsprojectother
supportcategoryintegrationsexternaltrackers supportcategoryintegrationsexternaltrackersbugzilla
supportcategoryintegrationsexternaltrackersibm
supportcategoryintegrationsexternaltrackersredmine
supportcategoryintegrationsexternaltrackersyoutrack
supportcategoryintegrationsexternaltrackerszentao
supportcategoryintegrationsexternaltrackersother supportcategoryintegrationsgmail
supportcategoryintegrationsgmailsetup supportcategoryintegrationsgmailerrors
supportcategoryintegrationsgmailquestions supportcategoryintegrationsgmailother
supportcategoryintegrationssourcegraph supportcategoryintegrationssourcegraphsetup
supportcategoryintegrationssourcegrapherrors supportcategoryintegrationssourcegraphquestions
supportcategoryintegrationssourcegraphother supportcategoryintegrationstrllo
supportcategoryintegrationstrllosetup supportcategoryintegrationstrlloerrors
supportcategoryintegrationstrlloquestions supportcategoryintegrationstrlloother
supportcategoryintegrationsvisualstudio supportcategoryintegrationsvisualstudiosetup
supportcategoryintegrationsvisualstudioerrors supportcategoryintegrationsvisualstudioquestions
supportcategoryintegrationsvisualstudioother supportcategoryintegrationsakismet
supportcategoryintegrationsakismetsetup supportcategoryintegrationsakismeterrors
supportcategoryintegrationsakismetquestions supportcategoryintegrationsakismetother
supportcategoryintegrationskroki supportcategoryintegrationskrokisetaup
supportcategoryintegrationskrokierrors supportcategoryintegrationskrokiquestions
supportcategoryintegrationskrokiother supportcategoryintegrationsmailgun
supportcategoryintegrationsmailgunsetup supportcategoryintegrationsmailgunerrors
supportcategoryintegrationsmailgunquestions supportcategoryintegrationsmailgunother
supportcategoryintegrationspim supportcategoryintegrationspimsetup
supportcategoryintegrationspimerrors supportcategoryintegrationspimquestions
supportcategoryintegrationspimother supportcategoryintegrationsrecaptcha
supportcategoryintegrationsrecaptchasetup supportcategoryintegrationsrecaptchaerrors
supportcategoryintegrationsrecaptchaquestions supportcategoryintegrationsrecaptchaother
supportcategoryintegrationsother supportcategoryintegrationsothersetaup
supportcategoryintegrationsothererrors supportcategoryintegrationsotherquestions
supportcategoryintegrationsotherother

1. Move on to Formatting options

1. This is kind of up to your flavour but below are some suggestions.

1. Next SLA breach

1. Priority

1. Preferred Region for Support

1. Subject

1. Organization

1. Assignee

1. Request date

1. Latest update by assignee

1. Change Order by to Next SLA breach, Ascending

1. Click Save

Congratulations you now have a personal Integrations Pod View.

SECTION: support/support-pods/kubernetes/_index.md

Purpose

A Support Pod for all things Kubernetes related: k8s, openshift, kas, agent, runner agent, k8s executor, CN Hybrid ref arch, helm, etc.

Current Objectives

- Collaborate on tickets
- Gain and share knowledge
- Improve documentation
- Identify bugs, produce issues and MRs
- Assist in updating relevant Support Training modules
- Serve as a resource for the greater Support team
- Influence product based on our experiences
- Develop tooling to aid in testing, troubleshooting, analysis, and learning
- Experiment with various deployment and architectural scenarios

Support Pod Members

- Lead: {{< member-by-name "Lewis Brown" >}} (@lwbrown)
- {{< member-by-name "Jessie Lee" >}} (@jessie)
- {{< member-by-name "Elif Munn" >}} (@emunn)
- {{< member-by-name "Aric Buerer" >}} (@abuerer)
- {{< member-by-name "Sam Bernal-Damasco" >}} (@definitelynotsam)
- {{< member-by-name "Lyle Kozloff" >}} (@lyle)
- {{< member-by-name "Andrew Winata" >}} (@awinata)
- {{< member-by-name "Mario Mora" >}} (@mmora)
- {{< member-by-name "Julian Dasmarninas" >}} (@jdasmarinas)
- {{< member-by-name "Brie Carranza" >}} (@bcarranza)
- {{< member-by-name "Christopher Mutua" >}} (@cmutua)
- {{< member-by-name "Michelle Almendarez" >}} (@malmendarez)

Collaboration Channels

- Slack channel: sptpodkubernetes
- Epic:
 - <https://gitlab.com/groups/gitlab-com/support/-/epics/145>
 - <https://gitlab.com/groups/gitlab-com/support/-/epics/191>

Open Office and Pairing Sessions

EMEA/AMER Pairing Session - Tuesday's at 15:30 UTC

Slack channels where k8s is frequently discussed

- supporttraining-k8s
- agile-architecture-cop
- aws
- aws-quickstart-guides
- cloud-native-strategy
- fagentforkubernetes
- fautodevops
- feedbackongitlabk8sagentworkshop
- kubernetes
- openshift
- reference-architectures
- sconfigure
- gdistribution
- grunner
- gitlabenvironmenttoolkit
- cs-questions

Learning Resources

SECTION: support/support-pods/licensing-and-renewals/_index.md

Purpose

Discuss and handle Licensing, Renewal and Subscription related matters for customers that involve technical or technical-adjacent problems and solutions.

Provide training, help and assistance with others who wish to grow their knowledge in the Licensing, Renewal and Subscription field of GitLab technical support. This is done via shadowing, pairing, and other methods that will be evolved and developed as we transition from just a group of engineers working the tickets and improving the processes to a proper Support Pod.

Current Objectives

- Collaborate on tickets
- Gain and share knowledge
- Improve documentation
- Identify bugs, product issues and MRs
- Assist in updating relevant Support Training modules
- Serve as a resource for the greater Support team
- Influence product based on our experiences
- Develop tooling to aid in testing, troubleshooting, analysis, and learning

Support Pod Members

- Lead: {{< member-by-name "Bethany McGrew" >}} (@jemcgrew)

- Members:

- {{< member-by-name "Ben King" >}} @benjaminking
- {{< member-by-name "Chantal Lawrence" >}} @chantalskye
- {{< member-by-name "Daphne Kua" >}} @dkua1
- {{< member-by-name "Donique Smit" >}} @doniquesmit
- {{< member-by-name "Keven Hughes" >}} @kevenhughes
- {{< member-by-name "Michelle Almendarez" >}} @malmendarez
- {{< member-by-name "Sameer Shaik" >}} @sshaik
- {{< member-by-name "Sokunrotanak Srey (Rotanak)" >}} @rotanak
- {{< member-by-name "Tom Houston" >}} @tom.houston
- {{< member-by-name "Tom McAtee" >}} @rrelax
- {{< member-by-name "Vidhya Hariharan" >}} @vidhya.h
- {{< member-by-name "Firdaws Farukh" >}} @ffarukh
- {{< member-by-name "Dima Serafin" >}} @dserafin-gitlab

Collaboration Channels

- Slack channel: supportlicensing-subscription

Open Office and Pairing Sessions

TBD

Learning Resources

Training Module

SECTION: support/support-pods/performance-and-reliability/_index.md

Purpose

A group of Support Engineers focused on the Performance and Reliability of GitLab and the systems it runs on.

Current objectives

- Collaborate on tickets related to Performance and Reliability
- Cultivate and propagate knowledge of Performance and Reliability in GitLab and how to troubleshoot Performance issues
- Collaborate on diagnostics tooling (scripts for reporting)
- Documentation updates

Support Pod members

- Lead: {{< member-by-name "Cody West" >}} (@cody)
- {{< member-by-name "Ulises Fierro" >}} (@ulisesf)
- {{< member-by-name "Nathaniel Rosario" >}} (@nrosario)
- {{< member-by-name "Edward Hilgendorf IV" >}} (@edwardhilgendorf)

- {{< member-by-name "Sam Bernal-Damasco" >}} (@definitelynotsam)
- {{< member-by-name "Greg Long" >}} (@glong)
- {{< member-by-name "Ryan Castro" >}} (@rcastro6)
- {{< member-by-name "Michael Lussier" >}} (@mlussier)
- and you

Collaboration channels

- Slack channel: sptpodperformance
- Epic - <https://gitlab.com/groups/gitlab-com/support/-/epics/145>

SECTION: support/support-pods/runner/_index.md

Purpose

Provide a space to collaborate and for Support Engineers to get help on Runner.

Scope

Everything runner related. Compliments CI/CD.

Current objectives

- Gain and share knoweledge
- Documentation updates

Support Pod members

- Lead: {{< member-by-name "Justin Farmiloe" >}} (@jfarmiloe)
- Lead: {{< member-by-name "Tony Marsh" >}} (@tmarsh1)
- {{< member-by-name "Kenneth Chu" >}} (@kenneth)
- {{< member-by-name "Michael Trainor" >}} (@tmike)

Collaboration channels

- sptpodrunner
- (Bi)weekly meetings: TBD - if you'd like to lead, please add it to the Support calendar and mention it here!

Zendesk view

Since limitations in Zendesk prevent each pod from having a shared view, you will have to create one manually.

Follow the steps below and you should have a personal view in no time.

1. Click on Manage views
1. In the new Window on the top right click Add view
1. Give the view a name

1. Move to Conditions

1. Tickets must meet all of these conditions to appear in the view

1. Click Add condition and from left to right

- Status
- Less than
- Pending

1. Click Add condition and from left to right

- Tags
- Contains at least one of the following
- supportcategoryrunners supportcategorycategoryrunners supportcategoryrunnersshared

supportcategoryrunnerssharedinstallation supportcategoryrunnerssharedconfiguration

supportcategoryrunnerssharedquestions supportcategoryrunnerssharederrors

supportcategoryrunnerssharedothertopic supportcategoryrunnersdocker

supportcategoryrunnersdockerinstallation supportcategoryrunnersdockerconfiguration

supportcategoryrunnersdockerquestions supportcategoryrunnersdockererrors

supportcategoryrunnersdockerothertopic supportcategoryrunnersfreebsd

supportcategoryrunnersfreebsdinstallation supportcategoryrunnersfreebsdconfiguration

supportcategoryrunnersfreebsdquestions supportcategoryrunnersfreebserrors

supportcategoryrunnersfreebsdthertopic supportcategoryrunnershelm

supportcategoryrunnershelminstallation supportcategoryrunnershelmconfiguration

supportcategoryrunnershelmquestions supportcategoryrunnershelerrors

supportcategoryrunnershelmothertopic supportcategoryrunnersagent

supportcategoryrunnersagentinstallation supportcategoryrunnersagentconfiguration

supportcategoryrunnersagentquestions supportcategoryrunnersagenterrors

supportcategoryrunnersagentothertopic supportcategoryrunnersoperator

supportcategoryrunnersoperatorinstallation supportcategoryrunnersoperatorconfiguration

supportcategoryrunnersoperatorquestions supportcategoryrunnersoperatorerrors

supportcategoryrunnersoperatorothertopic supportcategoryrunnerslinux

supportcategoryrunnerslinuxinstallation supportcategoryrunnerslinuxconfiguration

supportcategoryrunnerslinuxquestions supportcategoryrunnerslinuxerrors

supportcategoryrunnerslinuxothertopic supportcategoryrunnersmacos

supportcategoryrunnersmacosinstallation supportcategoryrunnersmacosconfiguration

supportcategoryrunnersmacosquestions supportcategoryrunnersmacoserrors

supportcategoryrunnersmacosothertopic supportcategoryrunnerswindows

supportcategoryrunnerswindowsinstallation supportcategoryrunnerswindowsconfiguration

supportcategoryrunnerswindowsquestions supportcategoryrunnerswindowerrors

supportcategoryrunnerswindowsothertopic supportcategoryrunnersother

supportcategoryrunnersotherinstallation supportcategoryrunnersotherconfiguration

supportcategoryrunnersotherquestions supportcategoryrunnersothererrors

supportcategoryrunnersotherother

1. Move on to Formatting options

1. Update it to show the columns to your preference

1. Change Order by to Next SLA breach, Ascending

1. Click Save

SECTION: support/support-pods/sec/_index.md

Sec Pod is a technical interest Support Pod

focused on GitLab Sec section features.

Secure Pod members

- Lead: {{< member-by-name "Katrin Leinweber" >}} (@katrinleinweber-gtlb)
- Lead: {{< member-by-name "Brie Carranza" >}} (@bcarranza)
- Lead: {{< member-by-name "Duncan Harris" >}} (@duncanharris)
- Lead: {{< member-by-name "Kate Grechishkina" >}} (@kategrechishkina)
- Lead: {{< member-by-name "Christopher Chewa Mutua" >}} (@cmutua)
- {{< member-by-name "Mario Mora" >}} (@mmora)
- {{< member-by-name "Caleb Williamson" >}} (@calebw)
- {{< member-by-name "Ronnie Alfaro" >}} (@ralfaro)
- {{< member-by-name "Cleveland Bledsoe Jr" >}} (@cleveland)

Purpose, key results and exit criteria (if any)

Sec Pod is a way for Support Engineers interested in Sec section features to work on relevant tickets and projects together.

The goals of Sec pod are to:

- identify underlying patterns and trends across Sec tickets
- file targeted issues and detailed bug reports to improve our Sec features
- submit MRs to GitLab documentation for self-service support and ticket deflection
- assist customers and team members with problems and questions involving Sec section features

FAQ

How can I get involved in Sec Pod?

1. Talk with your manager.
1. Submit a merge request to add 'Support Focus: Secure' to your ZenDesk Groups in the Support Team data.
1. Let your teammates and groupmates know about your new focus area.
1. Join sptpodsec Slack channel.
1. Attend Sec Pod pairing sessions. (Check GitLab Support calendar for meeting times)

How the Sec Pod works

- In the sptpodsec Slack channel, we pin Slack messages about · tickets that we are keeping an eye on for colleagues, typically when they are out of the office.
 - During · pairing sessions, check the pinned messages to see if the tickets there require attention.
 - If you pin a ticket, please remove it when it no longer requires attention from the pod.
- We apply the scoped pod::secure label to the pairing issues that we create.

Support Pod Resources

- Weekly session: "Sec Pod Pairing" on the GitLab Support Team Calendar, currently scheduled Thursdays at 3:00 PM UTC.

- Slack channel: sptpodsec
- GitLab.com label: ~"devops::secure"

Sec Pod Troubleshooting Resources

Secure Stage

- Slack: ssecure
- GitLab.com label: ~"devops::secure"
- Features by group: <https://handbook.gitlab.com/handbook/product/categories/features/secure>
- Submitting a request for help: <https://gitlab.com/gitlab-com/sec-sub-department/section-sec-request-for-helphow-to-submit-a-request-for-help--to-the-sec-section-development-team>
- Analyzer code: <https://gitlab.com/gitlab-org/security-products/analyzers/>
- Container registries for analyzer images: <https://gitlab.com/security-products/>
- Scanner Report Schemas: <https://gitlab.com/gitlab-org/security-products/security-report-schemas/>
- 15.0 Readiness - Secure: <https://gitlab.com/groups/gitlab-com/support/-/epics/202>
- Secure & Protect terminology: <https://docs.gitlab.com/user/applicationsecurity/terminology/>
- Vulnerability Severity Levels: <https://docs.gitlab.com/user/applicationsecurity/vulnerabilities/severities/>
- Security Reports Examples: <https://gitlab.com/gitlab-examples/security/security-reports/>

SAST (Static Application Security Testing)

- Slack: gsecure-static-analysis
- GitLab.com label: ~"group::static analysis"
- Docs: <https://docs.gitlab.com/user/applicationsecurity/sast>
- Template: <https://gitlab.com/gitlab-org/gitlab/-/blob/master/lib/gitlab/ci/templates/Jobs/SAST.gitlab-ci.yml>
- Test/demo project: <https://gitlab.com/gitlab-com/support/test-projects/ci-examples/sast>
- Collection of separate demo project: <https://gitlab.com/gitlab-com/support/test-projects/ci-examples/secure/static-analysis>

Secret Detection

- Slack: gsecure-secret-detection
- GitLab.com label: ~"group::secret detection", ~"Category:Secret Detection"
- Docs: <https://docs.gitlab.com/user/applicationsecurity/secretdetection/>
- CI/CD template: <https://gitlab.com/gitlab-org/gitlab/-/blob/master/lib/gitlab/ci/templates/Jobs/Secret-Detection.gitlab-ci.yml>
- Secret scanner codebase: <https://gitlab.com/gitlab-org/security-products/analyzers/secrets/>
- Secret detection rules: <https://gitlab.com/gitlab-org/security-products/analyzers/secrets/-/blob/master/gitleaks.toml>
- Upstream project: <https://github.com/gitleaks/gitleaks>
- Test/demo projects: <https://gitlab.com/gitlab-com/support/test-projects/ci-examples/secret-detection/>

Dependency Scanning

- Slack: gsecure-composition-analysis

- GitLab.com label: ~"group::composition analysis"
- Docs: <https://docs.gitlab.com/user/applicationsecurity/dependencyscanning/>
- Template: <https://gitlab.com/gitlab-org/gitlab/-/blob/master/lib/gitlab/ci/templates/Dependency-Scanning.gitlab-ci.yml>
- Test/demo project: <https://gitlab.com/gitlab-com/support/test-projects/ci-examples/dependency-scanning/>
- Dependency List docs: <https://docs.gitlab.com/user/applicationsecurity/dependencylist/>

DAST (Dynamic Application Security Testing)

- Slack: gsecure-dynamic-analysis
- GitLab.com label: ~"group::dynamic analysis"
- DAST Docs: <https://docs.gitlab.com/user/applicationsecurity/dast/>
- DAST CI/CD template: <https://gitlab.com/gitlab-org/gitlab/-/blob/master/lib/gitlab/ci/templates/Security/DAST.gitlab-ci.yml>
- DAST API scanning: <https://docs.gitlab.com/user/applicationsecurity/dastapi/>
- DAST API scanning CI/CD template: <https://gitlab.com/gitlab-org/gitlab/-/blob/master/lib/gitlab/ci/templates/Security/DAST-API.gitlab-ci.yml>
- DAST API test/demo projects: <https://docs.gitlab.com/user/applicationsecurity/apisecuritytesting/example-dast-api-scanning-configurations>
- DAST test/demo projects: <https://gitlab.com/gitlab-org/security-products/demos/dast/>

IaC (Infrastructure as Code) Scanning

- Slack: gsecure-static-analysis
- GitLab.com label: ~"group::static analysis"
- Docs: <https://docs.gitlab.com/user/applicationsecurity/iacscanning/>
- CI/CD template: <https://gitlab.com/gitlab-org/gitlab/-/blob/master/lib/gitlab/ci/templates/Security/SAST-IaC.latest.gitlab-ci.yml>
- IaC Scanner Codebase: <https://gitlab.com/gitlab-org/security-products/analyzers/kics>

Security Dashboard / Vulnerability Report

- Slack: gsecurethreatinsights
- GitLab.com label: ~"group::threat insights"
- Security Dashboard Docs: <https://docs.gitlab.com/user/applicationsecurity/securitydashboard/>
- Vulnerability Report docs: <https://docs.gitlab.com/user/applicationsecurity/vulnerabilityreport/>
- Vulnerability pages docs: <https://docs.gitlab.com/user/applicationsecurity/vulnerabilities/>

Security Scan Policies

- Slack: gprotectcontainersecurity
- GitLab.com label: ~"devops::protect", ~"Category:Container Scanning", ~"group::container security",
- Scan policies overview: <https://docs.gitlab.com/user/applicationsecurity/policies/>
- Scan Results Policies Docs: <https://docs.gitlab.com/user/applicationsecurity/policies/mergerequestapprovalpolicies/>
- Scan execution policy docs:

<https://docs.gitlab.com/user/applicationsecurity/policies/scanexecutionpolicies/>

Code Quality

Technically owned by Secret Detection, but not related to security vulnerabilities.

- Slack: gsecure-secret-detection
- GitLab.com label: ~"Category:Code Quality"
- Docs: <https://docs.gitlab.com/ci/testing/codequality/>
- CI/CD Template: <https://gitlab.com/gitlab-org/gitlab/-/blob/master/lib/gitlab/ci/templates/Jobs/Code-Quality.gitlab-ci.yml>
- GitLab Code Quality Analyzer Codebase: <https://gitlab.com/gitlab-org/ci-cd/codequality>

Container Scanning

- Slack: gprotectcontainersecurity
- GitLab.com label: ~"devops::protect", ~"Category:Container Scanning", ~"group::container security"
- Docs: <https://docs.gitlab.com/user/applicationsecurity/containerscanning/>
- CI/CD template: <https://gitlab.com/gitlab-org/gitlab/-/blob/master/lib/gitlab/ci/templates/Security/Container-Scanning.gitlab-ci.yml>

License Scanning

- Slack: gsecure-composition-analysis
- GitLab.com label: ~"group::composition analysis"
- Docs: <https://docs.gitlab.com/user/compliance/licenseapprovalpolicies/>
- CI/CD template: <https://gitlab.com/gitlab-org/gitlab/-/blob/master/lib/gitlab/ci/templates/Jobs/License-Scanning.gitlab-ci.yml>

Fuzz testing

- Slack: gsecure-dynamic-analysis
- GitLab.com label: ~""
- API Fuzzing Docs: <https://docs.gitlab.com/user/applicationsecurity/apifuzzing/>
- Coverage Fuzzing docs: <https://docs.gitlab.com/user/applicationsecurity/coveragefuzzing/>
- API Fuzz test/demo projects: <https://gitlab.com/gitlab-org/security-products/demos/api-fuzzing/>
- Coverage Fuzzing test/demo projects: <https://gitlab.com/gitlab-org/security-products/demos/coverage-fuzzing/>

Learning Resources

- GitLab Certified Security Specialist
- GitLab Security Essentials
- Security Essentials Hands-on
- Support DAST Deep Dive

SECTION: support/support-pods/training/_index.md

Purpose

Ensure Support training modules and up-to-date, and meeting the needs of Support Engineering.

Current objectives

- Ensure all training modules have at least one maintainer
- Clean up the support-training project
- Train maintainers on how to maintain modules
- Develop & maintain training module templates

Support Pod members

- Co-lead: {{< member-by-name "Matthew Badeau" >}} (@mbadeau)
- Co-lead: {{< member-by-name "John Gaughan" >}} (@jgaughan)
- {{< member-by-name "Manuel Grabowski" >}} (@manuelgrabowski)
- {{< member-by-name "Cleveland Bledsoe Jr" >}} (@cleveland)
- {{< member-by-name "Jackson Bigelow" >}} (@jbigelow)
- {{< member-by-name "Michael Gibson" >}} (@mgibsongl)

Collaboration channels

- Slack channel - sptpodtraining
- Slack group - @spt-podtraining

Getting added to the @spt-podtraining Slack group

Use the Slack Requests issue template to get added to the @spt-podtraining group on Slack.

Useful Links

- TBD

.. Stay Informed

The Support Week in Review is a great way to remain informed. Also consider the following sources of information in order to remain aware of what's going on with Support Training:

TBD

SECTION: support/support-pods/upgrade/_index.md

Purpose

Support Pod for focus on all the aspects of Upgrading GitLab.

Current objectives

- Flush out what the upgrade pod will all be!
- <https://gitlab.com/gitlab-com/support/support-pods/-/issues/6>

Support Pod members

- {{< member-by-name "Davin Walker" >}} (@davinwalker)
- {{< member-by-name "Diana Stanley" >}} (@dstanley)
- {{< member-by-name "Nathaniel Rosario" >}} (@nrosario)
- {{< member-by-name "Sam Bernal-Damasco" >}} (@definitelynotsam)
- {{< member-by-name "Julie Martin" >}} (@juliemartin)
- {{< member-by-name "Justin Farmiloe" >}} (@jfarmiloe)
- {{< member-by-name Amaury "Mau" Yacksmith >}} (@myacksmith)
- {{< member-by-name "Wade Jenkins" >}} (@wwjenkins)
- {{< member-by-name "Manuel Grabowski" >}} (@manuelgrabowski)
- {{< member-by-name "Olivier Némox Saint-Dizier" >}} (@onemoz)

Collaboration channels

- Slack channel: sptpodupgrade
- Epic - <https://gitlab.com/groups/gitlab-com/support/-/epics/145>

SECTION: support/training/_index.md

Support Learning Pathways

All the learning pathways we have in Support are listed here. Each pathway contains a set of modules, and a module may reside in multiple pathways.

To create an issue for yourself with one of the module templates:

- Click on the module you want to pursue from the Skills Catalog page
- Title it as Your Name: Module Name
- Assign it to yourself
- Now you're all set to follow the instructions in the module!

NOTE: Contact your Onboarding Buddy if you have any questions about these pathways. Buddies are always happy to help!

Support hybrid model

The hybrid model is a structured training plan that will prepare you to work on GitLab.com, self-managed, and GitLab Dedicated support issues. You may find these checkpoints helpful while working through the hybrid model path.

```
<div class="x-scrollable">
<div style="width: 1800px;">
```

mermaid
sequenceDiagram
 participant Trainer
 participant New SE
 participant Manager

Manager->>New SE: Sets expectations for timeline and content.

Trainer->>New SE: Schedules sessions to cover specific topics and answer questions.

Note right of New SE: Starts SE Onboarding!

Note left of New SE: New SE completes their first public update

Trainer->>New SE: Provides assistance with the first pairing sessions. Gives feedback on public replies.

New SE->>Manager: Reviews progress and tickets with manager in 1:1.

Note left of New SE: Starts SaaS Basics!

Trainer->>New SE: Offers suggestions on tickets to answer and follow up on initial responses.

Manager->>New SE: Check progress, ensures new ticket responses are aligned with our expectations and coaches as needed.

loop Pairing Sessions

New SE->>New SE: Uses Pairing sessions as a tool to get help from other team members and to learn new topics
end

Trainer-->New SE: Schedules weekly follow up sessions.

Note right of New SE: Completes SaaS basics

Manager->>New SE: During 1:1 go over the SaaS onboarding and suggest topics for ongoing learning.

Note left of New SE: 6 weeks working on SaaS tickets allowing time to work on new topics during that time.

Note right of New SE: Starts Self-managed basics, focusing on the main differences with troubleshooting to make the most from the current knowledge.

Manager->>New SE: Set expectations for hybrid work split, starting with 90%/10%.

loop 1:1s

New SE->>New SE: As more experience is gained brings the topic to 1:1 looking to increase the % of Self Managed work by 10%. With a goal of ~50%/50%
end

</div>

</div>

Overview of Support Onboarding

When you first join the team everything will be new to you. Don't worry! In order to get you

started with GitLab quickly, apart from the company wide onboarding issue that will be assigned to you on Day 1 by PeopleOps, we also have an onboarding program in Support. There are two pathways:

1. Support Engineer Onboarding Pathway
1. Support Manager Onboarding Pathway

NOTE: You can start the PeopleOps onboarding and the Support Onboarding at the same time, or you can complete PeopleOps and then switch to Support. Determine the best path with your manager.

Support Engineer Onboarding Pathway

Your primary Support Engineer Onboarding Issue is created from the New Support Team Member Start Here template. This issue keeps track of your completion of the onboarding modules shown in the table below.

It is recommended that you start the modules in the order listed, but it depends on your learning style. Note that the specific learning pathway modules are not included in the list, and when you open and start working on those modules may differ from other team members as well.

For example, many Support Engineers on the "GitLab Support Learning Pathway" will open the "Working on Tickets" module to start pairing with others once they have parts of the various basics module completed. Similarly, prior experience with administering a self-managed GitLab instance may change the order you complete the modules as well. If you are unsure about how best to approach onboarding, please discuss it with your manager.

Typically, for a new team member in Support, completion of the below onboarding modules (including those in the Area of Focus Pathway) takes approximately 6 weeks. A new team member's first ticket reply ranges typically in their first 28 - 38 days (week 3 - 5 of Support onboarding).

Module	Duration	Description
-----	-----	-----
Git & GitLab Basics	2 days	Understand our products and services
Customer Service Skills	2 Days	Understand how we interact with customers, and how to utilize your customer service skills to ensure customer success
GitLab Support Basics	1 Day	Understand how GitLab Support operates and the most common workflows
Zendesk Basics	1 Day	Utilize ZenDesk to perform ticket management
Triageing Tickets	4 Days	Understand how to triage tickets including the triage view in Zendesk
Customer Calls	6-12 hours	Understand when and how to organize and lead successful calls with customers
Documentation	1 Day	Become proficient in making documentation and merge requests

Choosing your initial Area of Focus

Work with your manager to decide your initial Area of Focus.

1. For GitLab (SaaS or Self-managed), complete the GitLab Support Learning pathway.
1. For License & Renewals, complete the License and Renewals Learning Pathway.

GitLab.com Administration access

Whenever GitLab.com administration access is required to complete any of the modules listed on this page, the training will direct you to the GitLab-com Admin module.

GitLab Support Learning Pathway

The GitLab Support pathway covers all platforms that we offer. In completing these modules, Support Engineers are expected to answer tickets submitted to any of them.

NOTE: As described in Support hybrid model, engineers typically complete one Area of Focus, and then add another. Please discuss your timeline with your manager.

Module	Duration	Description
Working on Tickets	2 Weeks	Help customers by pairing with Support Engineers and replying to tickets
GitLab-com SaaS Basics	2 Weeks	Understand the basics to answer GitLab.com (SaaS) product related tickets
GitLab Installation & Administration Basics	1 Week	Understand the different ways in which GitLab can be installed and managed
Self-Managed Support Basics	2 Weeks	Understand the basics to answer Self Managed product related tickets
GitLab Dedicated Basics	1 Week	Understand the basics to answer GitLab Dedicated related tickets

When these modules are complete:

1. Let your manager know that you are ready to proceed to the appropriate on-call rotation training.
1. With your manager, discuss how to describe your Area of Focus percentage in the support-team project, removing onboarding.

License and Renewals Learning Pathway

Module	Duration	Description
License and Renewals	2 Weeks	Understand the basics to answer our product's License and Renewals related tickets
Working on Tickets	2 Weeks	Work concurrently with your L&R module -- Help customers by pairing with Support Engineers and replying to tickets

On-call rotations

After you complete an Area of Focus, talk with your manager about joining an on-call rotation. Usually, you complete only one of these modules, and join only one on-call rotation.

Module	Duration	Description
GitLab.com CMOC	1 Day	Understand the responsibilities of being the Communications Manager On Call (CMOC) for an active GitLab.com incident
Customer Emergencies	1 Week	Understand the responsibilities of being on-call for Customer Emergencies

Support Engineer Achievable Progress - First 6 Months

Our onboarding pathway gives new Support Engineers an opportunity to learn at their own pace and explore. We strongly believe in learning by doing (70/20/10 learning model), and encourage Support Engineers to start contributing on tickets (with public or internal comments) from as early as your 3rd week.

NOTE: The 6 months starts after your initial Support Learning Pathways are complete.

The following reference table can be used as a guideline on achieving comfort with ticket management in your first 6 months in GitLab Support. Managers can also use this table to gauge the progress of new hires in their first 6 months after onboarding.

NOTE: Table data is shown by month - for instance, in my 4th month as a SaaS-focus support engineer, I can aim to make about 60+ comments on tickets per month.

Month	Public Comments per month Reference Range - Self-managed	Public Comments per month Reference Range - SaaS
1	0-10	0-20
2	10-30	20-40
3	30-50	40-60
4	50+	60+
5	60+	80+
6	70+	100+

Support Manager Onboarding Pathway

Your primary Support Manager Onboarding Issue is based on the New Support Team Member Start Here template. This Issue keeps track of and completes the onboarding modules shown in the following table.

It is recommended that you complete the modules in the order listed, unless an issue mentions that you can start something else simultaneously. Typically, for a new manager in Support, completion of the below onboarding modules should take 3 weeks.

Module	Duration	Description
Support Manager Basics	2 Weeks	Understand support management processes and workflows, including on-call rotations
Git & GitLab Basics	1 Week	Understand our products and services

GitLab Support Basics	1 Day	Understand how GitLab Support operates and the most common workflows
ZenDesk Basics	1 Day	Utilize ZenDesk to perform ticket management
Triage Tickets	4 Days	Understand how to triage tickets including the triage view in Zendesk
Customer Emergencies	1 Week	Understand the responsibilities of being on-call for Customer Emergencies
SSAT Reviewing for Managers	1 day	Understand how to handle Support Satisfaction feedback results

When this pathway is complete, let your manager know that you are ready to join the appropriate on-call rotations. (Your Support Manager Basics issue contains the instructions for this step.)

As always, please submit an Issue or MR if you have suggestions on how this page could be better!

SECTION: support/training/onboarding_buddy.md

You've become an onboarding buddy to a new Support Engineer. Congratulations! You've been granted a pivotal role in helping someone new learn about how to best contribute to our customers, our team, and our product.

Responsibilities

These responsibilities of the Support onboarding buddy are an extension of the GitLab Onboarding Buddy responsibilities, so please review that page first.

1. Help the new Support Engineer through any sticking points they have in their onboarding modules.
1. Communicate often with the new hire, and either help (or direct them to get help) when stuck.
1. Help the new Support Engineer adapt to and learn the role, by having frequent calls and pairing sessions to learn about our day-to-day duties.
1. Encourage the new Support Engineer to work tickets that push them out of their initial comfort zone.
1. If you will be out of office in the first 2 weeks, or have numerous commitments, consider discussing with your manager about either having someone else take on the role or to help clear some of your commitments, so that you can make sure you are present and can commit to the role.

Structure

During your time as onboarding buddy, you'll be working alongside the new Support Engineer while they complete their onboarding. Consider the timeframes below to be estimations: the pace will really be determined by the new Support Engineer.

| Week (estimated) | Task |

| ----- | ----- |

| Weeks 1-2 | Call to meet the new Support Engineer, help them get acclimated, answer any

general questions they may have, discuss schedules, and share anything else they may find useful as they get started. |

| | Open the lines of communication in Slack. Throughout this process, check in with them periodically to see how their onboarding module is progressing and ask if they need help.

|

| | Call to help them update the team page, GitLab Sandbox Cloud for GCP, the GDK, or anything else they may have questions about or trouble with. |

| Weeks 3+ | Pair at least twice, but more often if you can. Ideally, aim for once a week. |

| | Invite them to shadow you on any customer calls. |

| | Identify tickets that help push the engineer out of their comfort zone, and encourage the engineer to self-assign these tickets. |

Ideas & Suggestions

Note that everyone's needs are different, so consider the following as a list of ideas to customize to their individual needs. Not everyone needs everything from the below lists, but everyone would find at least a few of the following helpful.

First 1-2 weeks

- Meet and get to know each other
- Answer general questions
- Talk about your "schedule" and how you work every day - walk them through a few example days
- Talk about how to use tools like Slack, Navan, etc.
- Show them some helpful Slack channels
- Which Slack channels to keep track of daily
- Explain the SWIR and options for staying informed, like digest issues (and label subscriptions for a "newsletter experience"), the sptswir Slack channel, or the audio edition
- Show them some helpful handbook pages to read during onboarding
- Show them the GitLab architecture diagrams
- Show them some product or support-team-meta issues and clarify that they can contribute to anything
- Remind them they can get reimbursed for any books or training, and show them the Spending Company Money page. If it's expensive, talk to their manager first.
- Show them our Testing Environments, and introduce them to GitLab Sandbox Cloud for GCP (which replaced support-resources)
- Help them update the GitLab team page with their info (one of their Onboarding Issue checklist items)

Pairings

- Introduction to Zendesk and how we use it:
 - Walk them through Zendesk and how to use it
 - Answer any questions they have about the ticket workflow handbook pages
 - Walk through your own process for choosing and answering tickets
 - Talk about setting salutations in signature
 - Show them how to create a ticket pairing issue
 - While trying to help them feel comfortable, we also want new SEs to develop the low level of shame value by going out of their comfort zone. Stress that there are no stupid questions (maybe share the most recent "silly question" that you asked) and encourage them to ask

questions in different public Slack channels

- Have them setup pairings with different subject matter experts when they feel stuck on a topic

- Pair on tickets:

- Share your screen and answer a few easy tickets with them
- If they brought any tickets to the call, answer those first
- Alternate who shares their screen and answers the tickets
- Reproduce problems whenever you can. Show them how to quickly spin up an instance using your preferred method (Docker, GitLab Sandbox Cloud for GCP, etc.)

- Customer calls:

- Ping them when you have a customer call so they can shadow it
- Show them how to find customer calls to shadow
- Be available for their first customer call, if possible. If not, do what you can to ensure they have a shadow to help them out

What to do afterwards

- Over the first few months, if you have anything interesting you're working on that you think they can learn from, ping them and see if they'd like to either pair or shadow you on it.

SECTION: support/training/onboarding_hybrid_path_checkpoints.md

Support Hybrid Model Learning Path Checkpoints

Checkpoints for new starters who are working through their support hybrid learning path. This is a guide only, progress is not linear.

Learn about GitLab

- You understand GitLab's mission, vision, values, and culture
- You understand what is expected of you in your role and what it means to be a successful support engineer
- You've met most of your region (if you're in EMEA or APAC) in the Support Team call
- You've met up for 'coffee' with some of your teammates
- You understand how to make edits to a handbook page
- You've completed your compliance tasks in the Onboarding Issue
- You understand how we use Calendars and the different meeting types

Learn about Your Support Team

- You've met with more teammates for 'coffee'
- You have submitted a new update to the handbook
- You understand who our customers are and why they use GitLab
- You are getting to know what the GitLab suite offers its customers
- You are getting to know how customers can get started with GitLab
- You may have completed and closed your Onboarding Issue

- You are establishing your learning pathway through the GitLab modules with the New Team Member Start Here module
- Pairing sessions setup with Onboarding buddy:
 1. Pairing session fundamentals - what's the point, what you get out of it, etiquette of pairing session
 1. Explaining ZD views and how to action things

Working in Support at GitLab

- You are attending crush sessions and started pairing for your training modules
- You understand how to triage, assign and provide a first response on a ticket
- You understand the types of customer tickets that are being worked on by support engineers
- You understand what your role as a support engineer entails
- You know which teams and roles outside of Support work closely with Support Engineers
- You have completed your New Support Team Member Start Here module
- You have submitted MRs
- You have created issues

Contributing Back to Your Peers and the Team

- You have completed:
 - GitLab-com SaaS Account Basics (associate and intermediate only)
 - GitLab-com SaaS Basics
- You have taken assignment of a couple of SaaS Account tickets
- You have shadowed some customer calls
- You understand how to ask for help through the different channels
- You know which views/dashboards to work from within Zendesk
- You understand the organizational structure of GitLab

Leading Customer-Facing Video Interactions

- You have started/working through these modules:
 - Omnibus GitLab training module
 - GitLab.com console
 - License & Renewals module
- You have led customer calls

Refining Your Documentation

- You can triage/solve SaaS, Self-Managed, Dedicated and L&R tickets - your initial Support Learning Pathway is complete!
- You have confidence contributing to crush and pairing sessions
- You have gotten to know more folks outside your region or outside of Support
- You have started/working through the Documentation module

Participate in Upgrade Assistance

- You have been DRI on upgrade assistance tickets
- You are comfortable initiating customer calls as necessary
- You have updated your Areas of Focus in the Support Team Page

- You have discussed which on-call roster you will be working towards (CMOC or Customer Emergencies) with your manager (does not apply to Associate)

On-call, Onboarding Buddy and Technical Assessment Interview Rotations

- You have begun shadowing for addition to the on-call roster for Customer Emergencies or CMOC (does not apply to Associate)
- You are ready to be nominated as an Onboarding buddy for new team members
- You are working towards completion of the Technical Assessment Interview module
- You have scheduled in The 90-Day Checkin discussion where several of your onboarding checklist items are reviewed
- You are achieving (or working closely towards) your First 6 months achievable progress

SECTION: support/training/transition_training_module_gitlab_learn.md

This document covers the process for transitioning existing GitLab Support training modules to GitLab Learn.

Requisites

- Complete the GitLab Learning Evangelist training.

Transition template

The following template shows how each element in a Training Module aligns with GitLab Learn components.

mermaid

graph TD;

X(Transition Template)-->A

X-->C

X-->E

E(Tasks)-->F(GitLab Learn - SmartCards)

C(Training Module)-->D(GitLab Learn - Pathways)

A("Support Learning Objective")-->B(GitLab Learn Journey);

B-->Z(GitLab Learn)

D-->Z

F-->Z

Transition process

Step 0 - The Handbook Page

Every training requires its content to be in the handbook; the only exception are trainings with internal use only content.

Step 1 - Define transition goals

The first step in the transition is to visualize how the training will work on GitLab Learn, what new components will be used, and which limitations you might have.

The key elements to consider in GitLab Learn are:

1. Quizzes

At the moment, only single or multi-select answer questions are available.

1. Project type SmartCards

This smartcard allows the author to request a URL to be given. The functionality is useful for requesting and verifying an issue, such as an access request or pairing issue.

Step 2 - Create an issue with L&D team for the new pathway

- Create an issue with the L&D team using the new pathway template.

Step 3 - Create your Pathway

A Training module becomes a pathway in GitLab learn.

- Create your pathway

The pathway is similar to the issue used on Training Modules.

- Define collaborators

Collaborators will be allowed to modify the pathway settings and add smartcards to the pathway.

- Set the pathway to private

This setting is needed during the transition. The final setting may be different depending on the audience for the training module.

- Create a badge, so members who complete the material will have it in their profile.

Step 3 - Create the Smartcards

Tasks become smartcards in GitLab Learn.

- Select the option to edit the pathway and from there create the smartcards.
- Any smartcard type can be used.
- Quizzes are optional but recommended. Depending on how long the training can be, we can have

one per section or one halfway through and one in the end.

- Make sure to set the smartcard as locked, so any smartcard will require the previous one to be completed.
- Be creative! GitLab Learn offers different smartcard types and also gives room for other tools by allowing iframes with html.

Step 4 - Test Run

As a collaborator or author, the pathway cannot be fully tested. For this step, we will require volunteers to help us test our pathway.

- Maintain the Pathway private and publish it.
- Add the reviewers in the Share field for the pathway.
- Share the pathway URL with the reviewers.

What does success look like ?

- No smartcards can be skipped.
- The quizzes have the correct answers.
- By completing the pathway it will show 100% completed.

Step 5 - Request final review from the L&D team and publish

- Put a comment in the issue created from the first step to request a final review.
- Once the pathway has been approved and depending on the audience, change it to public.

Considerations

- Collaboration in GitLab Learn won't be as flexible as in GitLab, and will depend on Evangelists or Curators for final updates.
- The handbook is the single source of truth (SSOT). For this reason, any change to the training must start in its handbook page.
- A pathway author will be considered as DRI, and must ensure important updates in the training are also present in GitLab Learn as soon as possible.

SECTION: support/workflows/2fa-removal.md

Overview

This workflow focuses on disabling Two-factor Authentication (2FA) on a GitLab.com account. The general principles for authenticating a request are covered in our account verification workflow.

2FA removal and other account actions can only be completed if the workflow below is successful.

Related topics

GitLab Team Members

If the user is a GitLab team member, have them contact IT Ops.

2FA removal within GitLab

Self Service 2FA removal

In most cases, users can disable 2FA themselves and regain access to their accounts using one of the documented methods.

> As of August 2020, Support will not intervene for free users if self-service methods do not work for them.

Enterprise Owner 2FA removal for Enterprise users

A top-level group owner can disable 2FA for any enterprise user who is also a group member. With the rollout of the `enterpriseusersautomaticclaim` feature flag, users are automatically marked as an enterprise user if a group has a verified domain, and the user's primary email matches a verified domain.

Definitions

- Account holder: The person who uses the account day-to-day. The individual themselves may or may not be the account owner.
- Enterprise owner: One or more people who represent the business entity who purchased a paid plan with GitLab, hold Owner permissions in that paid namespace, and have a verified domain.

Conditions for SaaS users

A SaaS user must meet one of the following conditions to be eligible for a 2FA reset.

1. The user occupies a seat in a paid group on GitLab.com, or a top-level group owner intends to add the user to the paid group.
1. The user is an Enterprise User.
1. The user is the primary billing contact on a current invoice for a SaaS purchase.
1. A GitLab team member (account managers, CSMs, or others) collaborates with the holder of this account in an account management project.
1. The user account is required for SSO access to Customers Portal to manage a paid subscription - see: Conditions for 2FA Reset when account is used to access Customers Portal.

More succinctly: they're paid, they use the account to pay, or we use the account to communicate with them.

In many cases, a top-level group owner may submit a ticket on behalf of the user. See the Account verification matrix for more information.

Account verification matrix

Find the Account verification matrix on the Account Owner Verification handbook page.

Conditions when account is used to access Customers Portal

Customers Portal requires all customers to access through a Linked GitLab Account.

The user is eligible and 2FA can be reset when one of following conditions are met:

1. The request is made by the primary billing contact on the latest invoice for a GitLab subscription.
1. The GitLab account is linked to the customers portal account for the primary billing contact on the latest invoice for a subscription purchase.

If an invoice can not be provided, suggest sign in with legacy email/password, where an invoice can be downloaded.

Keep the Ticket simple and accurate

Because 2FA removal tickets are a matter of record, the ticket must be simple, accurate, and tightly focused on the access issue.

Do not allow the customer to bring up unrelated topics.

Disable 2FA with support intervention

Support intervention for 2FA removal after the above steps have been attempted is only possible for users with an existing paid plan when the ticket is created. For security purposes, Support will not process 2FA resets for users who are added to a paid subscription for the express purpose of having 2FA disabled on their account.

Workflows

Request for 2FA removal initiated by the account holder

Requests initiated by the account holder will be prompted by an autoreply to provide information to satisfy the security challenges.

Step 0: Validation

Some initial validation steps will occur automatically:

- Email / Username match
- Group membership validation
- Group subscription validation

If any of these are inaccurate, the ticket will be closed.

If a user submits a 2FA reset request ticket using the 2FA Assistance category but does not use the 2FA removal ticket subcategory, set the form subcategory to 2FA removal. If the ticket is obviously about 2FA reset but the customer didn't use any of the 2FA ticket categories, use the General::Forms::Incorrect form used macro to have Support Operations take the appropriate action on the ticket. If the user is not eligible for support, the ticket will automatically close.

Step 1: Checking challenge answers

> Note: In case the user sends back very minimal information and it's clear it's not sufficient or the answers are vague, reply asking for more information immediately after their response. You can provide some additional guidance, such as "please provide the exact date and time of the commit, not just an approximate one".

1. To verify the challenge answers use the Zendesk GitLab User Lookup App or, for those who have admin access, check at <https://gitlab.com/admin/users/USERNAME>.
1. Use the ZenDesk GitLab Super App's 2FA Helper to determine the risk factor (GitLab internal) based on the user's answers. Data classification criteria and any notes are in the Internal Handbook - Data Classification table (GitLab internal), which is considered the source of truth. If you need to leave a comment manually (instead of through the app), use the `Support::SaaS::Gitlab.com::2FA::2FA Internal Note` macro to put an internal note on the ticket.
 - Challenge answers must be evaluated against a paid namespace if the user is a member of any paid namespace. If the user is not a member of a paid namespace, refer to Conditions for 2FA Reset Consideration for further guidance.

1. If verification passed: Request that your decision be peer-reviewed by another member of the team via Slack [supportgitlab-com](#). They will perform the steps in 2a.
1. If the verification failed: Move to step 2b.

Step 2a: User successfully proves account ownership

This section is typically done by the peer reviewer. If needed, the peer reviewer (or approving manager) may leave an approval note, in which case the original reviewer will perform the actions.

1. If you agree with the decision, sign into your admin account and locate the username in the users table or by going to <https://gitlab.com/admin/users/usernamegoeshere>
 1. Under the account tab, click Edit, add an Admin Note, and save.
 1. On the account tab, click on Disable 2FA.
 1. Use the `Support::SaaS::Gitlab.com::2FA::2FA Removal Verification - Successful` macro.

Step 2b: User fails to prove account ownership

> Note: Do not provide hints to answers, or let the user know which challenges they got right or wrong. That is how social engineering works!

1. If the user is unable to pass the risk factor:
 1. Inform them that without verification we will not be able to remove 2FA, but they may request an Enterprise Owner create a request on their behalf (if they would qualify), use the `Support::SaaS::Gitlab.com::2FA::2FA Removal Verification - GitLab.com - Failed - Final Response` macro.
 1. Mark the ticket as "Solved".

Request for 2FA removal initiated by an Enterprise owner

Requests initiated by an Enterprise owner should include a Support PIN.

Note that Zendesk performs the following checks when a ticket is submitted for 2FA removal initiated by an Enterprise owner:

1. Has support entitlement?
1. Requester email domain matches target email domain?
1. Requester is owner of top-level paid namespace?
1. Target is a member under the top-level paid namespace?

If any of those fail, then the user is sent the regular 2FA challenges. Use step 1b below in this case.

If other challenges are sent, note that owners should answer the challenges in reference to their own account. Other answers are not acceptable.

Step 0: Validation

Some initial validation steps will occur automatically:

- Email / Username match
- Group membership validation
- Group subscription validation

If any of these are inaccurate, the ticket will be closed.

Step 1a: Verifying Support PIN

> Note: In case the user sends back very minimal information and it's clear it's not sufficient or the answers are vague, reply asking for more information immediately after their response.

1. If the user was sent regular challenge answers, use step 1b instead.
1. Use the ZenDesk GitLab Super App's 2FA Helper (as of May 1, 2025) to determine the risk factor (GitLab internal) based on the user's answers. Data classification criteria and any notes are in the Internal Handbook - Data Classification table (GitLab internal), which is considered the source of truth. If you need to leave a comment manually (instead of through the app), use the Support::SaaS::Gitlab.com::2FA::2FA Internal Note macro to put an internal note on the ticket.
1. To verify the Support PIN use admin access and check at <https://gitlab.com/admin/users/USERNAME>.
1. Since the user was sent the Support PIN response, the other required conditions are already matched:
 - Requester is an Enterprise owner
 - Target is an Enterprise user
1. If verification passed: Request that your decision be peer-reviewed by another member of the team via Slack supportgitlab-com. They will perform the steps in 2a.
1. If the verification failed: Move to step 2b.

Step 1b: Checking challenge answers

> Note: In case the user sends back very minimal information and it's clear it's not sufficient

or the answers are vague, reply asking for more information immediately after their response.

1. Skip this step if the user was sent the Support PIN autoresponse in Zendesk.

1. To verify the challenge answers use the Zendesk GitLab User Lookup App or, for those who have admin access, check at <https://gitlab.com/admin/users/USERNAME>.

1. Use the ZenDesk GitLab Super App's 2FA Helper to determine the risk factor (GitLab internal) based on the user's answers. Data classification criteria and any notes are in the Internal Handbook - Data Classification table (GitLab internal), which is considered the source of truth. If you need to leave a comment manually (instead of through the app), use the Support::SaaS::Gitlab.com::2FA::2FA Internal Note macro to put an internal note on the ticket.

- Challenge answers must be evaluated against a paid namespace if the user is a member of any paid namespace. If the user is not a member of a paid namespace, refer to Conditions for 2FA Reset Consideration for further guidance.

1. If verification passed: Request that your decision be peer-reviewed by another member of the team via Slack supportgitlab-com. They will perform the steps in 2a

1. If the verification failed: Move to step 2b

Step 2a: Enterprise Owner successfully proves their identity

This section is typically done by the peer reviewer. If needed, the peer reviewer (or approving manager) may leave an approval note, in which case the original reviewer will perform the actions.

1. If you agree with the decision, sign into your admin account and locate the username in the users table or by going to <https://gitlab.com/admin/users/usernamegoeshere>

1. Under the account tab, click Edit, add an Admin Note, and save.

1. On the account tab, click on Disable 2FA.

1. Use the Support::SaaS::Gitlab.com::2FA::2FA Removal Verification - Successful macro.

Step 2b: Enterprise Owner fails to prove their identity

> Note: Do not provide hints to answers, or let the user know which challenges they got right or wrong. That is how social engineering works!

1. If the user is unable to pass the risk factor:

1. Send the additional challenges with the Support::SaaS::GitLab.com::2FA::Additional 2FA Challenges macro.

1. You can ask the owner to generate a Support PIN and provide it to us in a response to the ticket, as an owner vouch. NOTE: If another user is CC'd on the ticket, once you've verified the Support PIN ask the user to generate a new PIN to revoke the previous one.

1. You may also leverage Backup methods for authenticating an owner.

1. If the user is still unable to pass the risk factor:

1. Inform them that without verification we will not be able to remove 2FA, use the Support::SaaS::Gitlab.com::2FA::2FA Removal Verification - GitLab.com - Failed - Final Response macro.

1. Mark the ticket as "Solved".

Backup methods for authenticating an owner

If a group owner does not include the owner vouch, you may use another method to verify their identity. It must be an action that has been specifically instructed by Support and identifiably unique to the situation. Some examples include having the owner:

- create a private snippet, containing the ticket number as a string.
- create an issue in a project they have access to with a specific piece of text that you provide.
- create a new project at a path that you provide.

Large Customers

{{< alert type="note" >}}

Note: This process is being deprecated on April 30, 2025. All customers using this process have been notified.

{{< /alert >}}

For customers who are large enough to have an account management project, a different workflow can be configured for them that will allow Support to more easily disable 2FA for any of their users that require it. Before this process can be used, a GitLab team member from either Customer Success or Sales must perform a few setup steps (described below). If a customer requests this workflow, please refer them to either of those individuals.

Setup (for CS & Sales)

The steps to follow depend on whether or not the customer has a shared Slack channel with us. Either the customer's Customer Success Manager (CS) or Account Manager (Sales) is responsible for performing this setup. Please proceed to Shared Slack Channel if they do or No Shared Slack Channel if they don't.

Method 1: Shared Slack Channel

1. Find out which users within the customer's organization are the ones that will be authorizing GitLab Support to disable 2FA on their users accounts. Obtain both the Slack handle and GitLab username of these users.
1. Create a file called 2FA Verification.md inside of the .gitlab/issuetemplates directory of the customer's Account Management project. If that directory does not exist, create it as well.
1. Populate the 2FA Verification.md file with the template below, taking care to replace the following variables from it with your specific customer's information:
 - CUSTOMERSLACKCHANNEL - The name of the shared Slack channel that the customer's organization has with us.
 - SLACKUSERNAME - The Slack handle of a user that is authorized to allow GitLab Support to disable 2FA for the customer's user accounts.
 - GITLABUSERNAME - The GitLab username of a user that is authorized to allow GitLab Support to disable 2FA for the customer's user accounts.

<details>

<summary markdown="span">2FA Verification Template</summary>

A user in your organization is requesting to have GitLab two-factor authentication removed from their account. Please review and complete the highlighted sections below.

Support Engineer Instructions

- Ping the customer's organization owners in CUSTOMERSLACKCHANNEL using the Notify Customer - Slack template. For this organization the owners are SLACKUSERNAME, SLACKUSERNAME, and SLACKUSERNAME.
- Fill out the Request Details section below.

Request Details

- User Requesting Reset: USERSGITLABUSERNAME
- Support Ticket: TICKETNUMBER

Customer Instructions

- Review the request and get in contact with the user requesting the reset to verify its authenticity.
- Comment on this issue indicating your approval.
- Unassign yourself and any others from this issue.
- Assign to the Support Engineer who opened this issue.

/assign GITLABUSERNAME GITLABUSERNAME GITLABUSERNAME, /label ~"2FA Reset" ~"Awaiting confirmation"

1. Create a Support Super form submission

- For "What is this request concerning?", select Modifications to a Zendesk Global Organization
- For "What kind of modification are you looking to make?", select Add 2FA exemption for large customers
- Fill out the other fields with the correct and relevant information it asks for

Method 2: No Shared Slack Channel

1. Find out which users within the customer's organization are the ones that will be authorizing GitLab Support to disable 2FA on their users accounts. Obtain the GitLab username of these users.

1. Create a file called 2FA Verification.md inside of the .gitlab/issuetemplates directory of the customer's Account Management project. If that directory does not exist, create it as well.

1. Populate the 2FA Verification.md file with the template below, taking care to replace the following variables from it with your specific customer's information:

- GITLABUSERNAME - The GitLab username of a user that is authorized to allow GitLab Support to disable 2FA for the customer's user accounts.

<details>

<summary markdown="span">2FA Verification Template</summary>

A user in your organization is requesting to have GitLab two-factor authentication removed from their account. Please review and complete the highlighted sections below.

Support Engineer Instructions

- Fill out the Request Details section below.

Request Details

- User Requesting Reset: USERSGITLABUSERNAME

-Support Ticket: TICKETNUMBER

Customer Instructions

- Review the request and get in contact with the user requesting the reset to verify its authenticity.
- Comment on this issue indicating your approval.
- Unassign yourself and any others from this issue.
- Assign to the Support Engineer who opened this issue.

/assign GITLABUSERNAME GITLABUSERNAME GITLABUSERNAME, /label ~"2FA Reset" ~"Awaiting confirmation"

1. Create a Support Super form submission

- For "What is this request concerning?", select Modifications to a Zendesk Global Organization
- For "What kind of modification are you looking to make?", select Add 2FA exemption for large customers
- Fill out the other fields with the correct and relevant information it asks for

Usage (for GitLab Support)

If a 2FA ticket is opened by an organization that has had this workflow configured for them, perform the following steps to process the request depending on whether or not the customer has a shared Slack channel with us.

>Note: 2FA removal for the user is approved by the Customer via the 2FA Verification template. This means the Customer will confirm with the User having 2FA removed and not support.

1. Create Issue

1. Open a new issue in the issue tracker of the customer's account verificati